Reader Monad

eader · Load+merge config.yaml, secrets.env, flags.json via Env · Build with Waqa

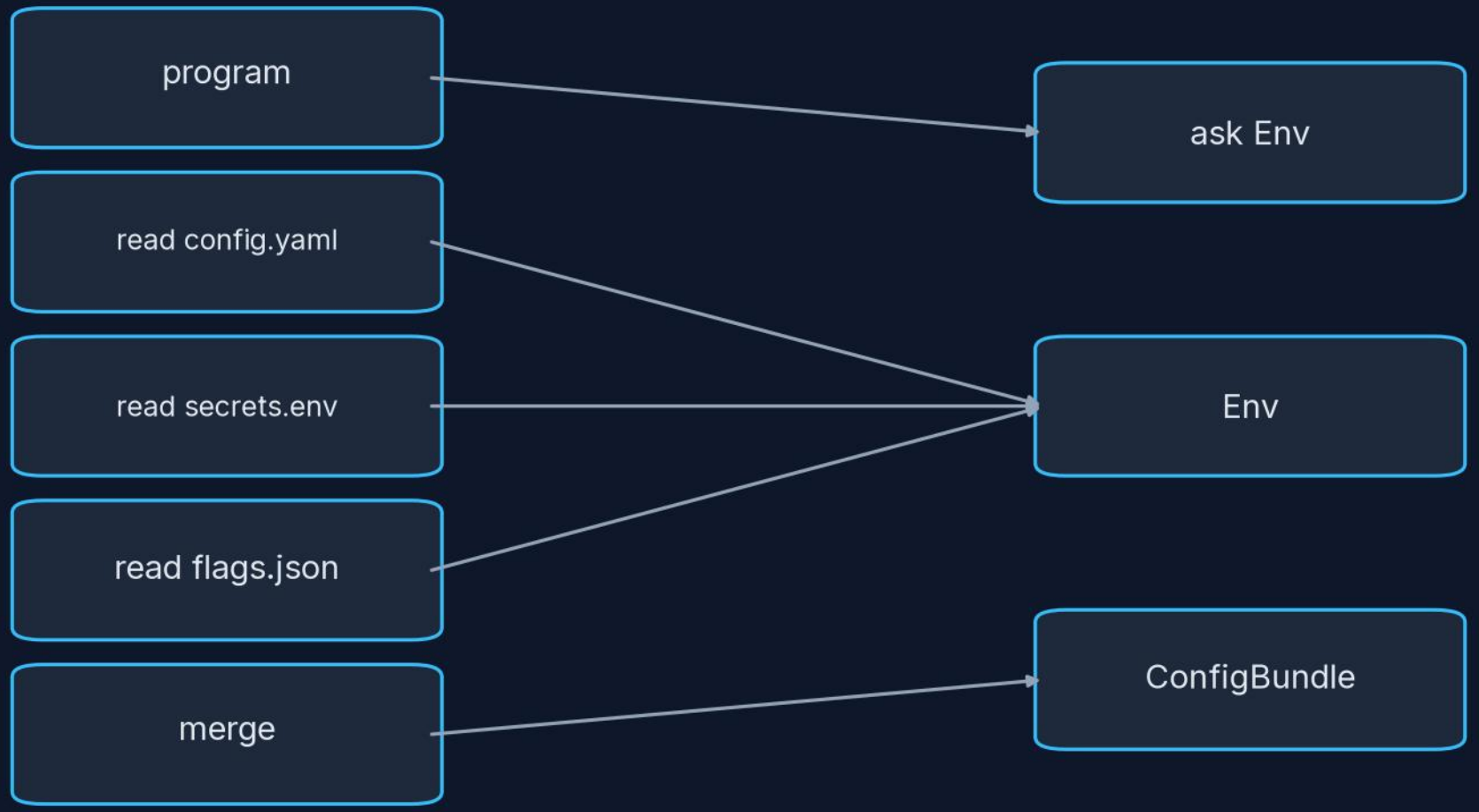
Today's Hook

- Problem: Load+merge config.yaml, secrets.env, flags.json via Env
- Pattern: Reader — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Load+merge config.yaml, secrets.env, flags.json via Env
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Reader: Load+merge config.yaml, secrets.env, flags.json vi

Real-World Scenario

- Running example: Load+merge config.yaml, secrets.env, flags.json via Env
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.functional.reader.problem;

import java.nio.charset.StandardCharsets;
import java.util.LinkedHashMap;
import java.util.Map;

/**
 * Naive: every helper takes fs/charset/baseDir explicitly — parameter drilling.
 * Easy to pass the wrong charset to one of three files.
 */
public final class NaiveMultiFileLoader {
    private final FileSystem fs;
    private final String baseDir;

    public NaiveMultiFileLoader(FileSystem fs, String baseDir) {
        this.fs = fs;
        this.baseDir = baseDir;
    }

    public ConfigBundle load() {
        var yaml = parseKv(read("config.yaml"));
        var secrets = parseKv(read("secrets.env"));
        var flags = parseKv(read("feature-flags.json"));
        return new ConfigBundle(yaml, secrets, flags);
    }
```

Naive Code (2)

```
    private String read(String name) {
        // "accidentally" hard-codes UTF-8 and ignores baseDir semantics
        return fs.readString(name, StandardCharsets.UTF_8)
                .orElseThrow(() -> new IllegalStateException("missing " + baseDir + "/" + name));
    }

    static Map<String, String> parseKv(String raw) {
        Map<String, String> m = new LinkedHashMap<>();
        for (String line : raw.split("\n")) {
            line = line.trim();
            if (line.isEmpty() || line.startsWith("#") || line.startsWith("{") || line.startsWith("}")) continue;
            line = line.replace("\"", "").replace(",", "");
            int eq = line.indexOf('=');
            int colon = line.indexOf(':');
            int sep = eq >= 0 ? eq : colon;
            if (sep > 0) {
                m.put(line.substring(0, sep).trim(), line.substring(sep + 1).trim());
            }
        }
        return m;
    }
```

Naive Code (3)

```
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.functional.reader.problem;

import com.buildwithwaqar.patterns.functional.reader.Reader;

import java.util.Map;

/**
 * Reader-based loader: Env (baseDir, charset, FileSystem, logger) is asked once
 * and threaded through composed reads — no parameter drilling.
 *
 * Nuance: Reader is not "a file reader API". It is a computation that depends on
 * an environment. Reading multiple files together is a great fit because every
 * step needs the same Env.
 */
public final class ReaderConfigLoader {
    private ReaderConfigLoader() {}

    public static Reader<Env, String> readFile(String relative) {
        return Reader.<Env>ask().flatMap(env -> Reader.pure(env.fs()
                .readString(relative, env.charset())
                .map(content -> {
                    env.info("read " + env.baseDir() + "/" + relative + " (" + content.length() + " bytes)");
                    return content;
                })
                .orElseThrow(() -> new IllegalStateException("missing " + env.baseDir() + "/" + relative))));
    }
```

Solution (2)

```
    public static Reader<Env, ConfigBundle> loadBundle() {
        return readFile("config.yaml").flatMap(yamlRaw ->
                readFile("secrets.env").flatMap(secretsRaw ->
                        readFile("feature-flags.json").map(flagsRaw -> {
                            Map<String, String> yaml = NaiveMultiFileLoader.parseKv(yamlRaw);
                            Map<String, String> secrets = NaiveMultiFileLoader.parseKv(secretsRaw);
                            Map<String, String> flags = NaiveMultiFileLoader.parseKv(flagsRaw);
                            return new ConfigBundle(yaml, secrets, flags);
                        })));
    }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.functional.reader.problem;

/** Problem: load config.yaml + secrets.env + flags.json with Env injected once. */
public final class ProblemDemo {
    public static void main(String[] args) {
        System.out.println("=== Reader problem: multi-file config merge ===");
        var fs = new FileSystem.InMemory()
                .put("config.yaml", "app=demo\nport=8080\n")
                .put("secrets.env", "DB_PASS=s3cret\n")
                .put("feature-flags.json", "beta=true\n");

        System.out.println("--- naive (paths + charset threaded everywhere) ---");
        System.out.println(new NaiveMultiFileLoader(fs, "/app").load());

        System.out.println("--- reader (Env asked once at the edge) ---");
        var env = Env.inMemory(fs);
        ConfigBundle bundle = ReaderConfigLoader.loadBundle().run(env);
        System.out.println(bundle);
        System.out.println("log=" + env.log());
    }
}
```

Walkthrough Output

```
=== Reader problem: multi-file config merge ===
--- naive (paths + charset threaded everywhere) ---
ConfigBundle[yaml={app=demo, port=8080}, secrets={DB_PASS=s3cret}, flags={beta=true}]
--- reader (Env asked once at the edge) ---
ConfigBundle[yaml={app=demo, port=8080}, secrets={DB_PASS=s3cret}, flags={beta=true}]
log=[read /app/config/config.yaml (19 bytes), read /app/config/secrets.env (15 bytes), read /app/config/feature-flags.json (10 bytes)]
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Reader.

Related Patterns

- Contrast Reader with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Reader → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.